

Learning Planar Object Pushing with Reinforcement Learning

ME5418 Final Project Proposal

Taowen Liu, A0352191J

Xuanwei Liu, A0357433B

Chengtao Zheng, A0351467Y

WHY:

- Pushing is a fundamental robotic manipulation skill that can be useful when directly grasping an object is difficult or unnecessary. For example, in industrial manipulation, a robot may first push a component to a desired position and orientation before grasping, assembling, or further manipulating it.
- Despite its simple appearance, pushing is a contact-rich manipulation problem. The resulting object motion depends on the contact location, pushing direction, object geometry, and physical properties such as friction. It can therefore be difficult to design a single hand-crafted controller that works reliably under different initial configurations.
- In this project, we investigate whether reinforcement learning can learn a closed-loop pushing policy that moves an object from a randomized initial pose to a desired target pose. We will primarily consider a T-shaped object similar to the Push-T task, while the same framework can also be evaluated on simpler object geometries.

CONVENTIONAL ALGORITHMS:

- Conventional robotic pushing approaches can use geometric planning, model-based control, trajectory optimization, or manually designed pushing strategies. Given an estimate of object pose and contact dynamics, these approaches can determine a sequence of pushing actions to move the object toward a target configuration.
- However, accurate contact dynamics can be difficult to model, especially when object geometry and friction vary. Model-based approaches may therefore require accurate physical parameters or repeated replanning.
- Instead, we would like to investigate whether a reinforcement learning agent can directly learn a feedback policy through interaction with the simulated environment, and adapt its pushing actions according to the current object and target poses.

PROBLEM STATEMENT:

- We consider a tabletop planar pushing environment containing a robot end-effector (or pusher), a movable rigid object, and a target pose. The main experiment will use a T-shaped object, although simpler shapes may also be considered.
- At the beginning of each episode, the object's initial position and orientation are randomly generated within the workspace. The target position and orientation may also be randomized.
- The robot must push the object from its initial configuration to the target configuration within a limited number of control steps. The task is completed successfully when both the position and orientation errors are below predefined thresholds.
- If time permits, we will additionally randomize physical properties such as the object-table friction coefficient to test whether the learned policy can remain robust under different dynamics.

RL CAST:

- **State space:**

The observation will contain low-dimensional robot and object states, including:

- current pusher/end-effector position;
- object position and orientation;
- target position and orientation;
- relative object-to-target position and orientation;

– optionally, object and robot velocities.

- **Action space:**

We will use a continuous action space controlling the planar motion of the pusher/end-effector. A simple action representation is

$$a_t = [\Delta x, \Delta y],$$

where Δx and Δy specify the desired displacement of the pusher during each control step.

If a robotic arm is used, these commands can instead be converted into end-effector Cartesian control commands.

- **Reward structure:**

The reward will mainly encourage progress toward the desired object pose. A possible reward is

$$r_t = -w_p d_{\text{pos}} - w_r d_{\text{rot}} + r_{\text{success}},$$

where d_{pos} is the distance between the current and target object positions, d_{rot} is the orientation error, and r_{success} is a large positive reward when the object reaches the target pose.

A small action or time penalty may also be included to encourage efficient solutions.

- **Neural Network Structure:**

Since the observation space is low-dimensional, we plan to use a simple multilayer perceptron (MLP) for the policy and value networks.

RL ALGORITHMS AND ENVISIONED RESULTS:

- We plan to use Proximal Policy Optimization (PPO) as the primary reinforcement learning algorithm, since it is widely used for continuous robotic control and is relatively stable and straightforward to train. Depending on the available time, we may additionally compare PPO with another continuous-control algorithm such as SAC.
- We will evaluate the learned policy mainly in terms of task success rate, final position error, final orientation error, and the number of control steps required to reach the target.
- The learned policy will be evaluated under different object initial poses and target poses. If time permits, we will further test different friction coefficients or object geometries to study the generalization and robustness of the policy.
- As a simple baseline, we will compare the RL policy with a heuristic pushing controller that moves the pusher toward the object and pushes it approximately in the direction of the target. We expect the learned policy to perform better on cases that require multiple contacts or simultaneous adjustment of object position and orientation.